RAG（Retrieval-Augmented Generation，检索增强生成）是一种把"外部知识库"和"大语言模型"结合起来的技术方案。

大语言模型本身存在三个固有问题：
第一，知识截止。模型的训练数据有截止日期，之后发生的事情它一概不知。
第二，私有知识缺失。你公司内部的文档、你个人的笔记，模型从未见过。
第三，容易幻觉。当模型不知道答案时，它倾向于"编一个看起来合理的答案"，而不是说"我不知道"。

RAG 的核心思路是：在模型回答问题之前，先从一份外部知识库里检索出相关内容，
把这段内容作为"参考资料"塞进 Prompt，让模型"看着资料回答"。

打个比方：模型是一个聪明的大学毕业生，知识面广但不知道你公司的具体情况。
RAG 相当于在提问前先递给他一份内部手册，让他看着手册回答你的问题。

RAG 的完整流程分为六步：
加载 → 切块 → 向量化 → 检索 → 拼 Prompt → 生成。

RAG 六步流程详解：

第一步：加载（Load）
把各种格式的文档（PDF、Word、Markdown、网页）转成纯文本。
PDF 是入门最常见的格式，用 pypdf 库可以逐页抽取文本。

第二步：切块（Chunking）
长文本不能整段塞给模型，一是超出上下文长度，二是检索粒度太粗。
通常切成 300~800 字的块，块之间保留少量重叠（overlap），避免语义被切断。
切块大小是 RAG 最容易出问题的地方：太大检索不精准，太小丢上下文。

第三步：向量化（Embedding）
用嵌入模型把每一块文本转成一串数字（向量），语义相近的文本向量距离近。
常用的中文嵌入模型有 BAAI/bge-small-zh-v1.5，只有约 100MB，本地就能跑。

第四步：检索（Retrieval）
用户提问时，把问题也转成向量，在向量库里找出距离最近的 top_k 个切块。
Chroma 是最常用的本地向量库，支持持久化。

第五步：拼 Prompt
把检索回来的切块拼成一段 context，写进 System Prompt，
要求模型"只根据 context 回答，无相关信息就直说不知道"。

第六步：生成（Generation）
把拼好的 Prompt 发给大模型（如 DeepSeek V3），得到基于知识库的回答。

RAG 常见误区：

误区一：以为 RAG 是"让模型变聪明"。
其实模型本身没变，只是每次提问时临时给它外挂一份资料。
资料换了，答案就跟着换，模型权重一动不动。

误区二：答错了就改 Prompt。
RAG 答非所问，90% 的情况是"检索没找对内容"，而不是"模型不会回答"。
正确做法：先打印检索回来的切块，确认它到底是不是相关片段，
不相关就调切块策略或换嵌入模型，相关但答错再去调 Prompt。

误区三：切块大小随便定。
500 字是中文的经验起点，不是标准答案。
技术文档可以小一点（300），叙述性文章可以大一点（800）。
没有一劳永逸的参数，要拿真实文档试。

误区四：一次检索就要答对。
多轮对话场景下，每轮都要重新检索，
且检索 query 只用"当前这一句问题"，不是把历史对话拼在一起。